

第4章 贪心算法

学习要点

- 理解贪心算法的基本原理

- 掌握贪心算法的基本要素

 - (1) 最优子结构性质

 - (2) 贪心选择性质

- 理解贪心算法与动态规划算法的差异

- 理解贪心算法的一般理论

例子：

 - (1) 活动安排问题
 - (2) 最优装载问题

 - (3) 单源最短路径
 - (4) 最小生成树

 - (5) 小数背包问题
 - (6) 哈夫曼编码

贪心算法基本思想

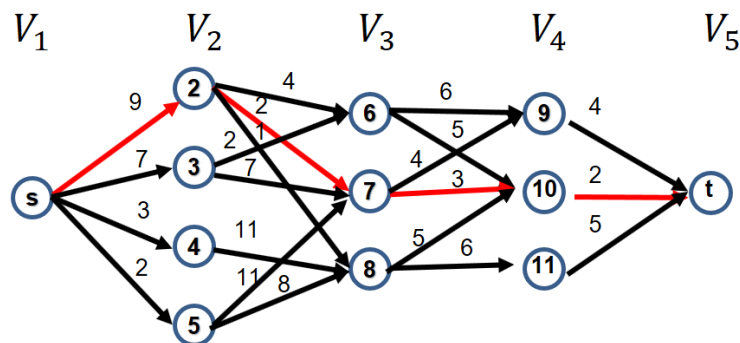
- 贪心算法是一个**分阶段决策过程**，在每个局部阶段，贪心法都做出一个**当前最优的局部决策**，并期望通过每次所做的局部最优决策产生一个**全局最优解**。
- 贪心算法类似于分治算法和动态规划，也是一种基于**子问题思想**的策略。

找零钱问题：假设有面值为5元、2元、1元、5角、2角、1角的货币，需要找给顾客4元6角现金，使付出的货币的数量最少？



贪心算法基本思想

- (1) 贪心算法总是作出在**当前看来最好的选择**。
- (2) 贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的**局部最优**选择。
- (3) 虽然贪心算法不能对所有问题都得到整体最优解，但对许多问题它能产生整体最优解。例如**单源最短路径问题**，**最小生成树问题**等。
- (4) 在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是**最优解的很好近似**。



多段图最短路径

贪心算法求解步骤

贪心算法的求解过程通常包括如下三个步骤：

- **分解**，将原问题求解过程划分为连续的若干个决策阶段
- **决策**，在每一个阶段依据**贪心策略**进行贪心决策，得到局部的最优解，并缩小待求解问题的规模
- **合并**，将各个阶段的**局部解合并**为原问题的一个**全局最优解**

贪心算法程序框架

```
Greedy(C) {           // C是问题的输入集合即候选集合
    S={ };             // 初始解集合为空集
    while (not Solution(S)) { //集合S没有构成问题的一个可行解
        x = Select(C);    //在候选集合C中做贪心决策
        S = S + {x};
        C= C - {Collection(x)}; //减去一个与x关联的集合，缩小问题规模
    }
    return S;
}
```

- ❑ 候选集合 **C** 是构造问题的解（包括最优解）的**对象集合**
- ❑ 解集合 **S** 表示**问题的解**，它随着贪心选择的进行不断扩展，直到构成一个满足问题的完整解
- ❑ 选择函数**Select**是**贪心策略**的实现过程，这是贪心法的关键，它指出哪个候选对象最有希望构成问题的最优解，选择函数通常和目标函数有关

活动安排问题

任务描述:

假设某社团某一天要组织 n 个活动 $E = \{1, 2, \dots, n\}$ ，其中每个活动都要求使用同一礼堂，而且在同一时间内只有一个活动能使用这个礼堂。每个活动 i 都有一个要求使用礼堂的起始时间 s_i 和结束时间 f_i ，且有 $s_i < f_i$ 。如果选择了活动 i ，则它在半开时间区间 $[s_i, f_i)$ 内占用资源。**若区间 $[s_i, f_i)$ 与区间 $[s_j, f_j)$ 不相交，则称活动 i 与活动 j 是相容的。**现在给定 n 个活动的开始时间和结束时间，请设计一个活动安排方案，使得安排的相容活动数目最多。

策略一：选择具有最早开始时间，而且不与已安排的活动冲突的活动。

$$E = \{(0, 10), (2, 5), (7, 9)\}$$



策略二：选择具有最短使用时间，而且不与已安排的活动冲突的活动。

$$E = \{(0, 4), (3, 6), (5, 9)\}$$



策略三：选择具有最早结束时间，而且不与已安排的活动冲突的活动。

实例分析

例：设待安排的11个活动信息如下表所示：

s[i]	1	0	3	3	5	5	6	8	8	2	12
f[i]	4	6	5	8	7	9	10	11	12	13	14

$S = \{[1, 4]\}$

s[i]	1	0	3	3	5	5	6	8	8	2	12
f[i]	4	6	5	8	7	9	10	11	12	13	14

$S = \{[1, 4], [5, 7]\}$

s[i]	1	0	3	3	5	5	6	8	8	2	12
f[i]	4	6	5	8	7	9	10	11	12	13	14

$S = \{[1, 4], [5, 7], [8, 11]\}$ $S = \{[1, 4], [5, 7], [8, 11], [12, 14]\}$

s[i]	1	0	3	3	5	5	6	8	8	2	12
f[i]	4	6	5	8	7	9	10	11	12	13	14

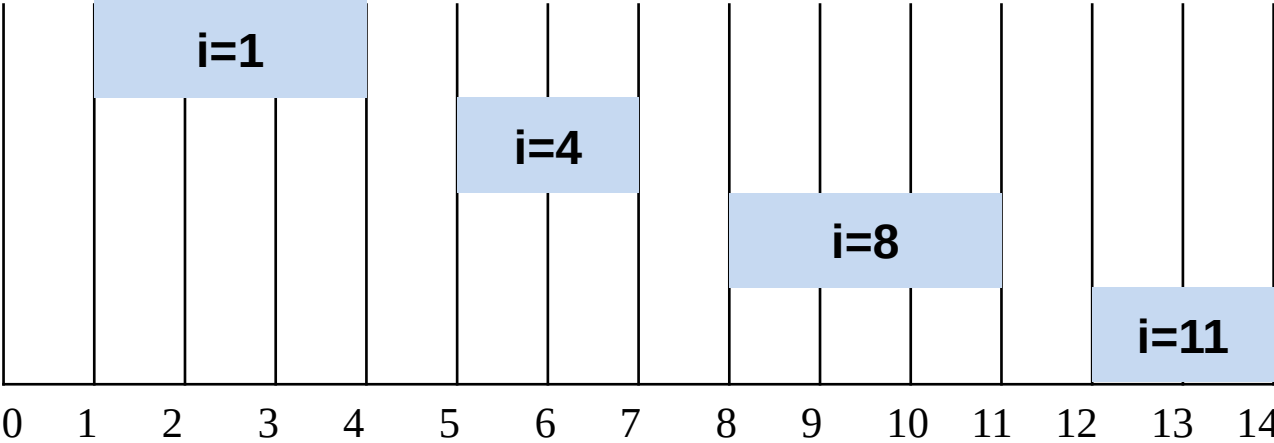
实例分析

例：设待安排的11个活动信息如下表所示：

s[i]	1	0	3	3	5	5	6	8	8	2	12
f[i]	4	6	5	8	7	9	10	11	12	13	14

排序后

i	1	2	3	4	5	6	7	8	9	10	11
s[i]	1	3	0	5	3	5	6	8	8	2	12
f[i]	4	5	6	7	8	9	10	11	12	13	14



算法思路

活动安排问题的贪心算法设计思路如下：

1. **预处理** 把所有的活动按照结束时间进行升序排列，有 $f[1] \leq f[2] \leq \dots \leq f[n]$
2. **选择第一个活动** 选择结束时间最早的活动 $E[1]$ ，当前决策 $selected = 1$
3. **贪心选择后续活动** 依次扫描后续的每一个活动 $E[i]$ ，如果 $E[i]$ 的开始时间**晚于**上一个选择的活动的结束时间，则安排当前活动 $E[i]$ ，令 $selected = i$ ；否则放弃 $E[i]$ 。

算法实现

```
int greedyEventSchedule(int n,int *timeStart,int *timeFinish) {  
    int i,j,selected,ans=0;  
    //冒泡排序，使得活动按结束时间升序排列  
    for(i=0; i<n; i++)  
        for(j=0; j+1<n; j++)  
            if(timeFinish[j]>timeFinish[j+1]) {  
                swap(timeFinish[j],timeFinish[j+1]);  
                swap(timeStart[j],timeStart[j+1]); //注意开始时间也需一致移动  
            }  
    selected = 0; //选择第一个活动  
    ans = 1;  
    for(i=1; i<n; i++)  
        if(timeStart[i]>= timeFinish[selected]) {  
            selected = i; //选择相容的最早结束活动  
            ans++;  
        }  
    return ans;  
}
```

$O(n^2)$

贪心算法正确性证明（数学归纳法）

□ 证明目标

$\forall n$ $p(n)$ 为真 // n 的论域为正整数集合

□ 证明框架

✓ 基础步骤: $p(1)$ 为真

✓ 归纳步骤: 证明 $\forall k (p(k) \rightarrow p(k+1))$

// 对任意正整数 k , 给出 $p(k) \vdash P(k+1)$ 的论证步骤

➤ 结论: 对任意正整数 n , $p(n)$ 成立

数学归纳法（实例）

猜测前 n 个奇数的求和公式，并证明之。

$$1=1$$

$$1+3=4$$

$$1+3+5=9$$

...

$$1+3+\dots+(2n-1) = n^2 \quad (n \text{ 为正整数})$$

证明：

基础步骤：

$$P(1) \text{ 为真, } 1=1^2$$

归纳步骤：

$$\text{对任意正整数 } k, P(k) \Rightarrow P(k+1)$$

$$1+3+\dots+(2k-1)+(2k+1) = k^2+2k+1 = (k+1)^2$$

因此， 对任意正整数 n , $P(n)$ 成立.



活动安排贪心算法的正确性证明

命题： 算法GreedySelector执行到第 k 步，选择的 k 项活动记为 $i_1 = 1, i_2, \dots, i_k$ ，那么**存在一个最优解包含** $i_1 = 1, i_2, \dots, i_k$ 。

证明（数学归纳法）：

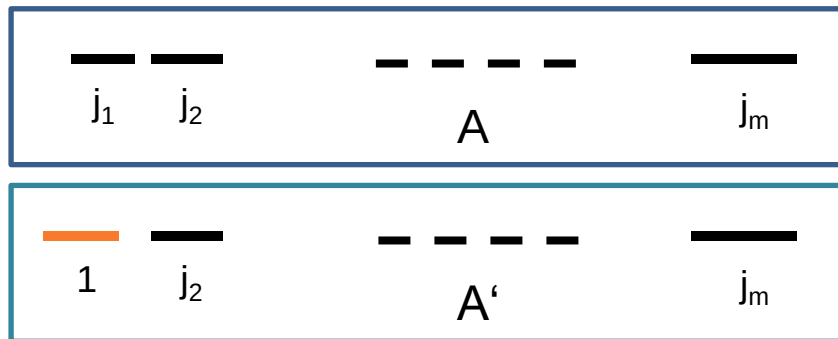
预处理 将集合 S 中的活动按照**结束时间递增顺序排列**，即记为 $S = \{1, 2, \dots, n\}$ ；假设 $A = \{j_1, j_2, \dots, j_m\}$ 是 S 的一个最优解，其中活动也按照**结束时间递增顺序排列**。

基础步： $k = 1$ 时选择活动为 $\{1\}$ ，需要证明存在一个最优解包含了活动 $\{1\}$

□ $j_1 = 1$ ，即最优解包含活动1

□ $j_1 \neq 1$ ，则用活动1替换最优解 A 中的 j_1 ，得到活动集合 A' ，既有： $A' = \{A - \{j_1\}\} \cup \{1\}$

A' 是相容活动集合，也是一个**最优**的活动安排



正确性证明

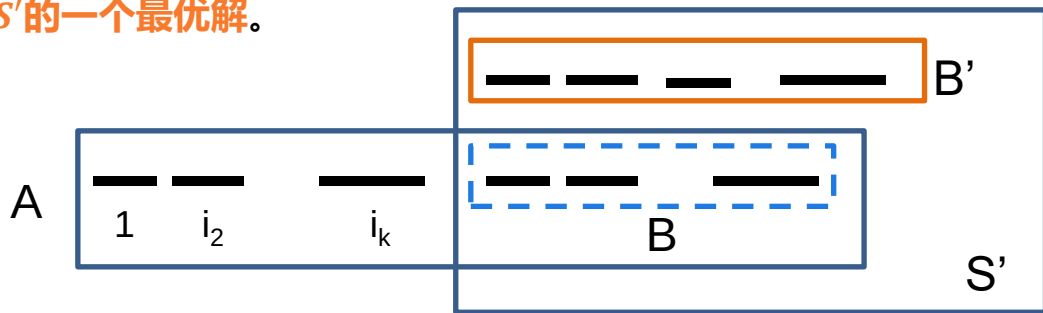
命题： 算法GreedySelector执行到第 k 步，选择的 k 项活动记为 $i_1 = 1, i_2, \dots, i_k$ ，那么**存在一个最优解包含** $i_1 = 1, i_2, \dots, i_k$ 。

证明（数学归纳法）：

归纳步： 假设对于正整数 k ，命题正确。令 $\{i_1 = 1, i_2, \dots, i_k\}$ 是GreedySelector算法前 k 步顺序选择的活动的，即存在一个最优解： $A = \{i_1 = 1, i_2, \dots, i_k\} \cup B$

假设 S' 是 S 中与 $\{i_1 = 1, i_2, \dots, i_k\}$ 相容的活动，即 $S' = \{j \mid s[j] \geq f[i_k], j \in S\}$

那么 B 是 S' 的一个最优解。



如果 B 不是最优解，则存在 B' 是 S' 的最优解，其活动数目比 A 多；

则构造 $A' = \{i_1 = 1, i_2, \dots, i_k\} \cup B'$ ，它是原活动集合 S 的相容活动安排，且活动数目比最优解 A 更多，**与前提矛盾！**

正确性证明

命题： 算法GreedySelector执行到第 k 步，选择的 k 项活动记为 $i_1 = 1, i_2, \dots, i_k$ ，那么**存在一个最优解包含** $i_1 = 1, i_2, \dots, i_k$ 。

证明（数学归纳法）：

归纳步： 假设对于正整数 k ，命题正确。令 $\{i_1 = 1, i_2, \dots, i_k\}$ 是GreedySelector算法前 k 步顺序选择的活动的，那么存在一个最优解： $A = \{i_1 = 1, i_2, \dots, i_k\} \cup B$

假设 S' 是 S 中与 $\{i_1 = 1, i_2, \dots, i_k\}$ 相容的活动，即 $S' = \{j \mid s[j] \geq f[i_k], j \in S\}$

那么 B 是 S' 的一个最优解。

对于子问题 S' **应用基础步的结论：** 即 S' 的结束时间最早的活动 i_{k+1} 包含在某一个最优解 B' 中，则可以构造原问题的一个最优解：

$$A' = \{i_1 = 1, i_2, \dots, i_k\} \cup B' = \{i_1 = 1, i_2, \dots, i_k, i_{k+1}\} \cup \{B' - \{i_{k+1}\}\}$$

关于贪心算法的基本思想，下列说法正确的是（）

- ☐ A 从全局最优出发，逐步做出局部选择
- ☒ B 在每一步都做出当前看起来最优的选择，希望最终得到全局最优解
- ☐ C 一定能保证得到全局最优解
- ☐ D 需要遍历所有可能解才能得到最优结果

提交

在经典的单资源活动安排问题中，为了安排尽可能多的相容活动，贪心策略通常选择（）

- ☐ A 开始时间最早的活动
- ☐ B 持续时间最短的活动
- ☒ C 结束时间最早的活动
- ☐ D 占用资源最少的活动

提交

在活动安排问题中，贪心算法之所以选用“结束最早”策略，核心原因是（）

- ☒ A 可以让剩余可用时间最大化
- ☐ B 实现最简单
- ☐ C 运行速度最快
- ☐ D 不需要排序

提交

小数背包问题

问题描述： 给定 n 种物品和一个背包。物品 i 的重量是 W_i ，其价值为 V_i ，背包的容量为 C ，应如何选择装入背包的物品使得装入背包中物品的总价值最大？这里，在选择物品 i 装入背包时，可以选择物品 i 的一部分，而不一定要全部装入背包。

输入： 多组测试数据。每组测试包括三行：第一行输入物品的总数 n ($n < 1000$) 和背包的容量 C ($C < 1000$)。第二行输入 n 个整数，表示物品的重量。第三行输入物品的价值。

输出： 输出装入背包的总价值，每组测试数据输出一行。

输入样例：

```
3 50
10 20 30
60 100 120
```

输出样例：

```
240
```

小数背包问题

小数背包问题数学模型:

$$\begin{aligned} \max \quad & \sum_{i=1}^n v_i x_i \\ \text{st.} \quad & \begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ 0 \leq x_i \leq 1, 1 \leq i \leq n \end{cases} \end{aligned}$$

0-1背包问题数学模型:

$$\begin{aligned} \max \quad & \sum_{i=1}^n v_i x_i \\ \text{st.} \quad & \begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0,1\}, 1 \leq i \leq n \end{cases} \end{aligned}$$

小数背包问题

策略一： 在不超出当前背包的剩余容量前提下，优先选择**价值最大**的物品，这样使得装入价值增长最快。

策略二： 在不超出当前背包的剩余容量前提下，优先选择**重量最轻**的物品，这样使得背包容量增长最慢。

策略三： 在不超出当前背包的剩余容量前提下，优先选择**价值率（价值除以重量）最大**的物品，这样使得背包中单位重量价值增长最快。

例： $n=3$, $c=20$, $v=(25, 24, 15)$, $w=(18, 15, 10)$

	(x_1, x_2, x_3)	$\sum w_i x_i$	$\sum v_i x_i$
策略一	$(1, 2/15, 0)$	20	28.2
策略二	$(0, 2/3, 1)$	20	31
策略三	$(0, 1, 1/2)$	20	31.5

小数背包问题

小数背包问题的贪心算法设计思路如下：

1. **预处理**，把物品按照**价值率**进行**降序排列**
2. **选择第一个物品** 根据贪心策略，首先选择**价值率最大**的物品，并记录该物品装入的重量。
3. **贪心选择后续活动** 依次扫描每一个物品，在没有超出背包容量的条件下，尽可能多地装入**当前价值率最高**的物品，并记录该物品装入的重量。

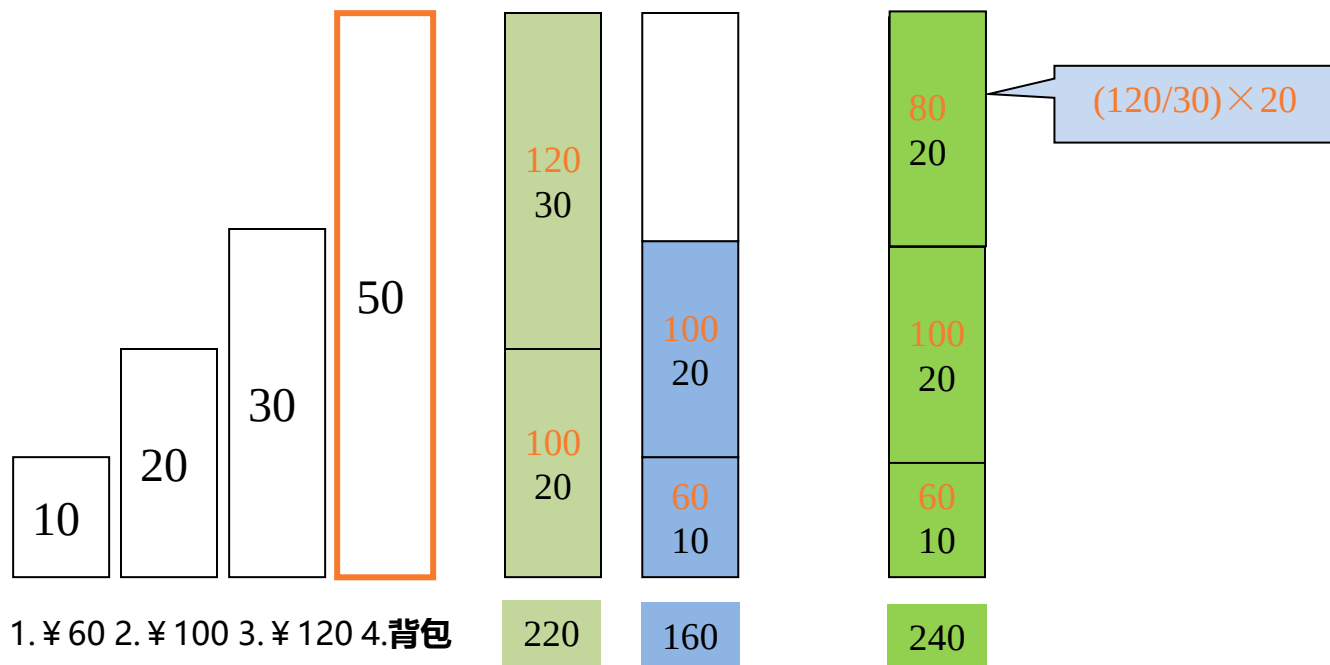
小数背包问题

```
#include<stdio.h>
#include<algorithm>
using namespace std;
#define MaxItems 1000
struct item{
    int weight;
    int value;
    bool operator <(const item& bb) const { //定义比较函数 (小于号)
        return value/(1.0*weight) > (1.0*bb.value)/bb.weight; //为什么乘以1.0?
    }
}; //定义物品的结构体
double greedyKnapsack(int n,int capacity,item* itemSet) {
    double ans = 0;
    sort(itemSet,itemSet+n); //STL中的快速排序算法
    for(int i=0; i<n; i++)
        if(capacity >= itemSet[i].weight) {
            ans += itemSet[i].value; //选择单价最大的物品
            capacity -= itemSet[i].weight;
        } else {
            ans += capacity*(itemSet[i].value*1.0)/itemSet[i].weight; //最后一个物品只能装部分
            break;
        }
    return ans;
}
```

$O(n \log n)$

0-1背包问题与小数背包问题

直观解释：0-1背包问题可能有**闲置空间**，摊薄平均利润率



贪心算法的基本要素



对于一个具体的问题，怎么知道**是否可用贪心算法**解此问题，以及**能否得到问题的最优解呢？**

2个重要的性质

贪心选择性质和**最优子结构性质**

贪心选择性质

□ 贪心选择性质：

指所求问题的**整体最优解**可以通过一系列**局部最优**的选择达到。这是贪心算法可行的第一个基本要素，也是贪心算法与动态规划算法的主要区别。

□ 与动态规划的区别：

- 动态规划以**自底向上**的方式解各子问题；
- 贪心算法以**自顶向下**的方式进行，并以迭代的方式作出相继的贪心选择，每作一次贪心选择就将所求问题简化为规模更小的子问题。

□ 对于一个具体问题，要确定它是否具有贪心选择性质，必须**证明每一步所作的贪心选择最终导致问题的整体最优解**。

最优子结构性质

- **最优子结构性质**：一个问题的最优解包含其子问题的最优解。
- 问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。**两类算法的共同点**



(1) 具有最优子结构的问题应该选用贪心算法还是动态规划算法求解？

➤ 均可。建议：前采用贪心解决问题（未必最优），后采用动态规划优化。

(2) 是否能用动态规划算法求解的问题也能用贪心算法求解？

➤ 未必（0-1背包问题 vs 小数背包问题）

0-1背包问题 vs 小数背包问题

0-1背包问题

给定 n 种物品和一个背包。物品 i 的重量是 w_i ，其价值为 v_i ，背包的容量为 C 。应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

对每种物品只有2种选择，即装入背包或不装入背包。不能将物品装入背包多次，也不能只装入部分的物品。

小数背包问题

与0-1背包问题类似，所不同的是在选择物品装入背包时，可以选择物品的一部分，而不一定要全部装入背包。

2个问题都具有最优子结构性质，极为相似。但小数背包问题可以用贪心算法求解，而0-1背包问题却不能用贪心算法求解

✓ 对于0-1背包问题，贪心选择不能得到最优解的原因：

➤ 其无法保证最终能将背包装满

➤ 部分闲置的背包空间降低单位背包空间的价值

✓ 0-1背包问题思想：比较选择该物品和不选择该物品所导致的最终方案，然后再作出最好选择。

➤ 由此就导出许多互相重叠的子问题。

➤ 这正是该问题可用动态规划算法求解的另一重要特征。

关于分数背包问题，正确的是？

- ☐ A 必须用动态规划解决
- ☒ B 贪心算法总能得到最优解
- ☐ C 需按物品价值降序装入背包
- ☐ D 以上都不对

提交

字符编码

一个包含100,000个字符的文件，各字符出现频率不同，如下表所示。请设计一个编码算法，把每一个字符转换为二进制编码。

不同的字符	a	b	c	d	e	f
频率(千次)	45	13	12	16	9	5
定长码	000	001	010	011	100	101
变长码	0	101	100	111	1101	1100

定长编码：用3位来表示一个字符，对整个文件编码需要**300,000**位。

变长编码：给频率高的字符较短的编码；频率低的字符较长的编码，达到整体编码减少的目的：

$$(45 \times 1 + 13 \times 3 + 12 \times 3 + 16 \times 3 + 9 \times 4 + 5 \times 4) \times 1000 = \mathbf{224,000}$$

二元前缀码

二元前缀码 在一个字符集的编码表（每一个编码都表示为0-1字符串）中，如果任何一个字符编码都**不是**其它字符编码的**前缀**，则该编码称为二元前缀码。

□ **编码**：把字符转换为二元前缀码的过程

□ **解码**：把二元前缀码（0-1符号串）映射为字符的过程。从第一个字符开始依次读入每个字符（0或者1），如果发现读到的子串与某个代码相等，就将这个字串译作相应代码的字符。

考虑字符集 $C=\{a, b, c, d\}$ ，如果其代码集合为 $Q=\{001, 00, 010, 01\}$ ，容易验证Q**不是**二元前缀码。考虑编码序列0100001，存在**两种解码**方法：

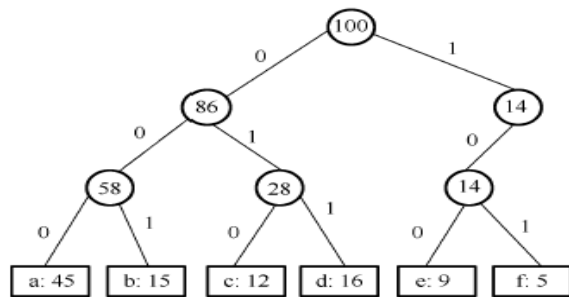
① 解码为 01, 00, 001，译码为**d, b, a**

② 解码为 010, 00, 01，译码为**c, b, d**

二元前缀码

- 二元前缀码的存储通常采用**二叉树**结构，令**字符保存在叶节点**，则该字符的前缀码对应根到该叶节点的一条**路径**。规定每个节点通向左儿子的边记为0，通向右儿子的边记为1，则**根到叶节点的路径**则可以转换为一个0-1串，映射为二元前缀码。
- 一个字符集可以构造**多个**二元前缀码。

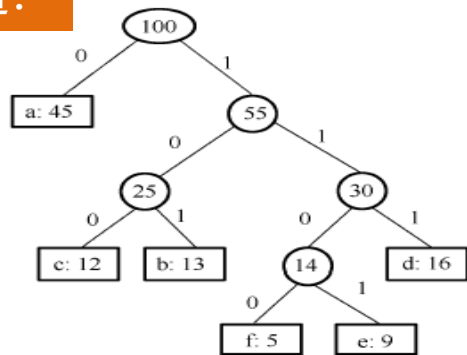
最优前缀码怎么定义，怎么构造？



a: 000
d: 011

b: 001
e: 100

c: 010
f: 101



a: 0
d: 111

b: 101
e: 1101

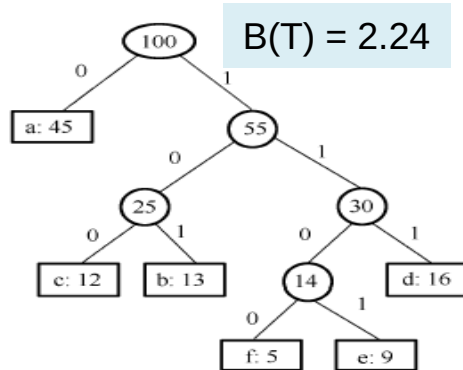
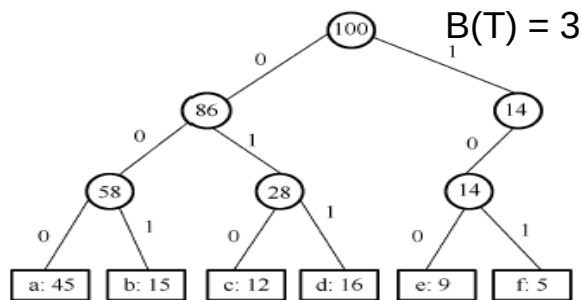
c: 100
f: 1100

最优前缀码

在一个文件中，字符集中字符出现的频次往往不尽相同。设 $C = \{x_1, x_2, \dots, x_n\}$ 是 n 个字符的集合， x_i 的频次是 $f(x_i)$, $i = 1, 2, \dots, n$ 。在一个二元前缀码中， x_i 的码长定义为其代码的**二进制位数**，或者二叉编码树中其对应**叶结点的深度**，记为 $d(x_i)$ 。则该编码方案的**平均码长**定义为：

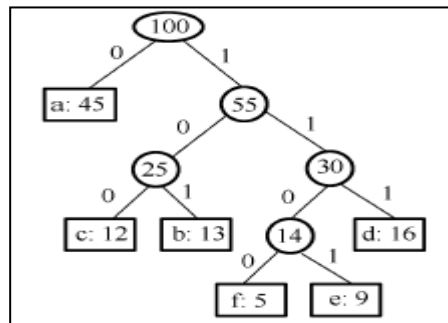
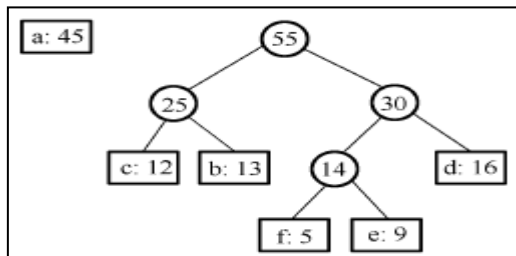
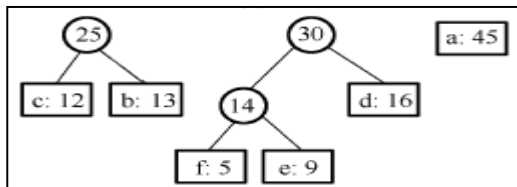
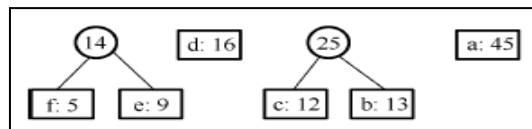
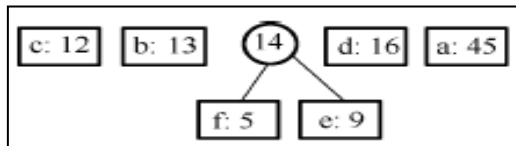
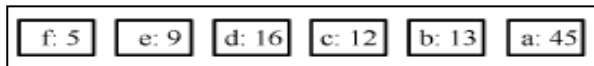
$$B(T) = \frac{\sum_{i=1}^n f(x_i) d(x_i)}{\sum_{i=1}^n f(x_i)}$$

最优前缀码则定义为**平均码长最小**的二元前缀码。



哈夫曼编码

- 哈夫曼编码算法的**贪心策略**是：**给频率高的字符较短的代码；频率低的字符较长的代码**
- 把编码映射成二叉树，则其贪心策略可表述为：**把频率高的字符分配给靠近根结点（较浅）的叶结点，把频率低的字符放置在远离根结点（较深）的叶结点**
- **自底向上构造**二叉编码树，由森林不断合并得到一棵二叉树



哈夫曼编码

为了便于找到频次最低的字符，哈夫曼算法建立一个以 f 为键值的优先队列 Q ，假设编码字符集中每一字符 c 的频率是 $f(c)$ 。哈夫曼编码算法以自底向上的方式构造最优编码树 T 。

- 每个字符构成一棵只包含一个结点的树，总共有 $|C|$ 棵构成一个森林。
- 合并频率最低的两棵树，并产生一棵新树，其频率为合并的2棵树的频率之和，并将新树插入优先队列 Q 。
- 循环此过程，经过 $n - 1$ 次这样的合并后，优先队列中只剩下一棵树，即最优二叉编码树 T 。

哈夫曼编码算法实现

```
struct cmp {
    bool operator () (const int &x,const int &y) {
        return x > y;
    }
}; //定义优先队列需要的比较函数
double haffmanCoding(int n, int* freq) {
    int i,total = 0,sumFreq = 0,joinFreq;
    priority_queue<int,vector<int>,cmp> heap; //优先队列，最小值优先
    for(i=0; i<n; i++) {
        total += freq[i]; //频次总和
        heap.push(freq[i]);
    } //形成优先队列
    while(heap.size()>1) { //循环选择队列中频次最少的两个元素合并
        joinFreq = 0; //合并后结点的频次
        for(i=0; i<2; i++) { //删除频次最少的两元素
            joinFreq += heap.top();
            heap.pop();
        }
        sumFreq += joinFreq;
        heap.push(joinFreq); //优先队列中插入合并结点
    }
    return sumFreq/(1.0*total); //返回平均码长
}
```

$$O(n \log(n))$$

贪心算法正确性证明（交换论证法）

交换论证法（Exchange Argument）：从任意一个最优解出发，经过不断用新的要素（符合贪心准则）**替换**解中的原有要素来改变这个解。通过有限步替换，把这个**最优解造成贪心算法的解**。如果替换过程中的每一步能保证解的**最优值不发生变化**，则证明了贪心算法的解也是最优的。

1. 给出贪心算法解A的描述
2. 假设O是一个**最优算法解**（如果有多个则随机选）
3. 找出O和A中的一个**不同要素**（可以是一个元素在O不在A或者反之，或者是一个子序列的顺序A和O不一样）
4. **交换**（Exchange）上述不同的要素，然后**论证**（Argue）得到的新解 O^* 不比O差
5. 论证类似的不同要素为**有限个**（多项式量级），然后**交换**有限次（多项式量级）可以消除所有的不同，同时保证了算法的质量不比O差

正确性证明

交换论证法原理：从任意一个**最优解**出发，经过不断用新的要素（符合贪心准则）**替换**解中的原有要素来改造这个解。通过有限步替换，把这个**最优解改变成贪心算法的解**。如果替换过程中的每一步能保证解的**最优值不发生变化**，则证明了贪心算法的解也是最优的。

1. **基础步：**给定任意一个最优解，根据贪心准则对最优解进行**改造**，也就是把第一步贪心选择的对象**替换**最优解中的特定要素，然后证明**替换后的新解也是最优解**
2. **递归步：**证明上述交换过程可以循环进行。也就是说，依次替换最优解的其他要素，直到新解的**每个分量**都符合贪心准则，并且证明**新解也是最优的**。

证明方法一般是：**最优子结构性质 + 基础步结论**

牛刀小试：删除数字问题

问题描述：给定一个高精度的正整数 n ($n \leq 240$ 位)，去掉其中任意 s 个数字后剩下的数字按原左右次序将组成一个新的正整数。对给定的 n 和 s ，寻找一种方案，使得剩下的数字组成的新数最小。

输入：多组测试数据。每组测试数据包括一行，输入 n 和 s 。

输出：最后剩下的最小数；每组测试数据输出一行。

输入样例：

```
178543    4
```

输出样例：

```
13
```

小试牛刀

有4个字符a~d，它们的频度分别为8，1，4，6。

问题：

(1) 构造其哈夫曼编码，并求平均码长是多少？

(2) 说明将“aabcabd”字符串编码结果。

解：(1) 哈夫曼编码：a: 0 b: 100 c: 101 d: 11

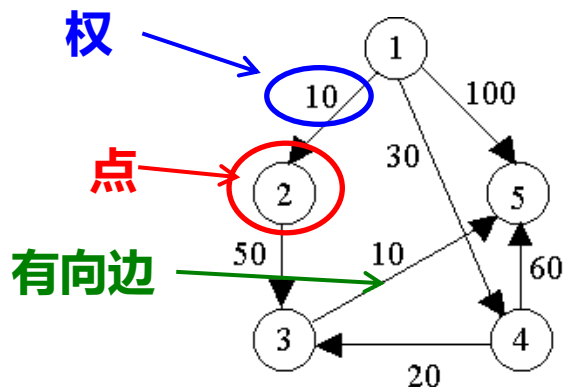
平均码长：1.84（或者 $35/19$ ）位/字符

(2) 0010010110011

单源最短路径

问题描述

给定带权有向图 $G = (V, E)$ ，其中每条边的权是非负实数。



$c(i, j)$: 节点 i 到 j 的距离

c	1	2	3	4	5
1	0	10	∞	30	100
2	∞	0	50	∞	∞
3	∞	∞	0	∞	10
4	∞	∞	20	0	60
5	∞	∞	∞	∞	0

给定 V 中的一个顶点，称为源。

注： ∞ (无穷大) 表示不可达

计算从源到所有其他各顶点的最短路长度。
(路的长度是指路上各边权之和)

单源最短路径-Dijkstra算法

Dijkstra算法是解单源最短路径问题的贪心算法

基本思想：

- 顶点集合 S ，初始为 $S = \{i\}$ ， S 不断采用**贪心选择**来扩充自身，直至 S 包含所有节点。
- 一个顶点属于集合 S 当且仅当从源到该顶点的最短路径长度已知。

特殊路径（受限路径）：设 u 是 G 的某一个顶点，把从源到 u 且中间只经过 S 中顶点的路称为从源到 u 的特殊路径，并用数组 $dist$ 记录当前每个顶点所对应的最短特殊路径长度。比如： $S = \{A, C\}$ ，那么 $A \rightarrow D$ ， $A \rightarrow C \rightarrow D$ 都是特殊路径， $A \rightarrow C \rightarrow F \rightarrow D$ 则不是。

基本步骤：

- (1) S 中仅含有源节点 i ；
- (2) 每次从 $V - S$ 中取出具有最短特殊路长度的顶点 u ，将 u 添加到 S 中，同时对数组 $dist$ 作必要的修改。
- (3) 一旦 S 包含了所有 V 中顶点， $dist$ 就记录了从源到所有其它顶点之间的最短路径长度。

Dijkstra算法

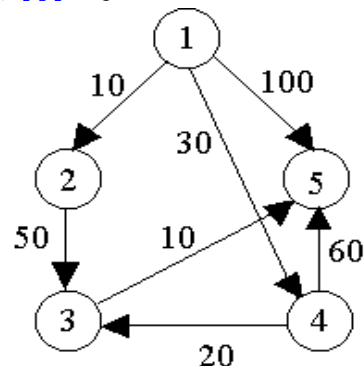
例：应用Dijkstra算法计算从源顶点1到其它顶点间最短路径。

数据结构：

$c(i, j)$ ：顶点 i 到 j 的距离（用于获得最优值）

$dist$ ：源到 j 的最短特殊路径长度

p ：节点 j 的先序节点（用于获得最优解）

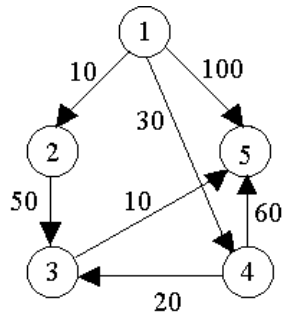


c	1	2	3	4	5
1	0	10	∞	30	100
2	∞	0	50	∞	∞
3	∞	∞	0	∞	10
4	∞	∞	20	0	60
5	∞	∞	∞	∞	0

j	1	2	3	4	5
$dist$	-	10	∞	30	100
p	-	1	0	1	1

Dijkstra算法

j	1	2	3	4	5
p	-	1	0	1	1



迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2

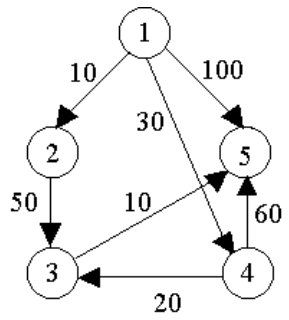
在 S 未包含的顶点中，选择特殊
路径长度最短的顶点加入到集合
 S

选择顶点2

Dijkstra算法

j	1	2	3	4	5
p	-	1	0	1	1

$p(2)$ 不变

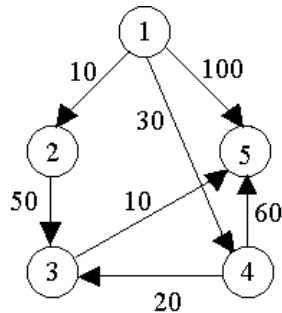


迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
初始	{1}	10	∞	30	100	2
1	{1, 2}	10				

更新集合S

源顶点1到顶点2的最短特殊路径长度不变。

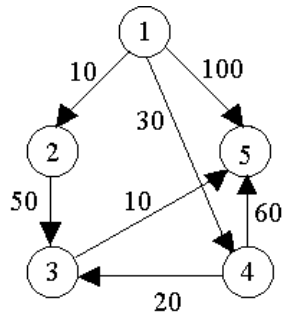
Q1: 源顶点1到顶点2是否会有更短路径?



j	1	2	3	4	5
p	1	1	0	1	1

迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
初始	{1}	10	∞	30	100	2
1	{1, 2}	10	?			
1到3之间是否可通过2中转? 即1至2的距离与2至3的距离之和是否更小?						

j	1	2	3	4	5
p	-	1	2	1	1

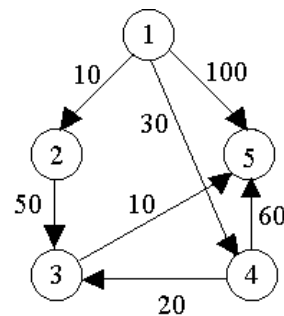


$d(1,3) = 60 < \infty$, 更新 $p(1,3) = 2$

迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2
2	{1, 2}	10	60			

以2为中间点计算

$$\begin{aligned}
 d(1,3) &= dist(1,2) + c(2,3) \\
 &= 10 + 50 = 60
 \end{aligned}$$



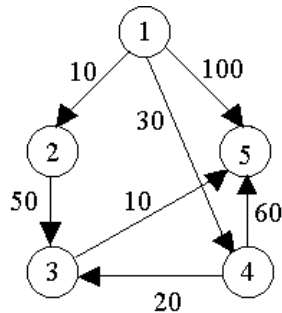
j	1	2	3	4	5
p	-	1	2	1	1

$d(1,4) = \infty > 30$, 不更新 $p(4)$

迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2
2	{1, 2}	10	60	30		

以2为中间点计算

$$d(1,4) = dist(1,2) + c(2,4) = 10 + \infty = \infty$$



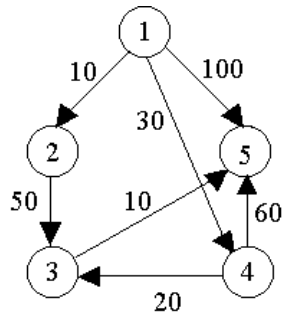
j	1	2	3	4	5
p	-	1	2	1	1

$d(1, 5) = \infty > 100$, 不更新 $p(5)$

迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2
2	{1, 2}	10	60	30	100	

以2为中间点计算

$$\begin{aligned}
 d(1, 5) &= dist(1, 2) + c(2, 5) \\
 &= 100 + \infty = \infty
 \end{aligned}$$



j	1	2	3	4	5
p	-	1	2	1	1

迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2
2	{1, 2}	10	60	30	100	4

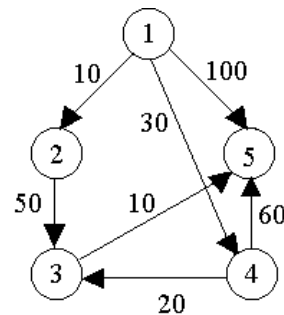
顶点2已在中,
不是备选项

选择S中未包含的最短特殊路径
值最小的顶点加入到集合S

 选择顶点4

Dijkstra算法

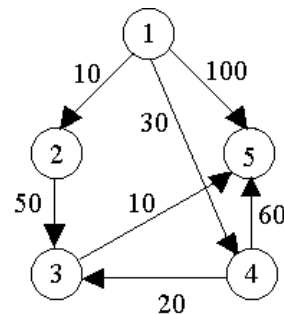
j	1	2	3	4	5
p	-	1	2	1	1



迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2
2	{1, 2}	10	60	30	100	4
3	{1, 2, 4}	10		30		
		源顶点1到顶点2和4的最短特殊 路径长度不变。				

Dijkstra算法

j	1	2	3	4	5
p	-	1	4	1	1

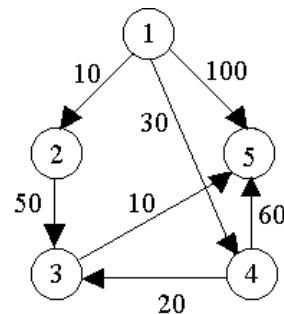


$d(1, 3) = 50 < 60$, 更新 $p(3) = 4$

迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2
2	{1, 2}	10	60	30	100	4
3	{1, 2, 4}	10	50	30		
		以4为中间点计算 $d(1, 3) = dist(4) + c(4, 3)$ $= 30 + 20 = 50 < 60$				

Dijkstra算法

j	1	2	3	4	5
p	-	1	4	1	4

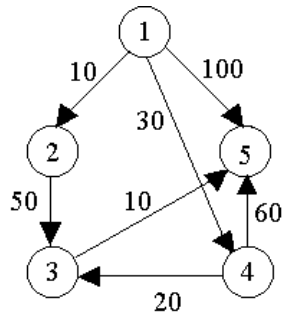


$d(1, 5) = 90 < 100$, 更新 $p(5) = 4$

迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2
2	{1, 2}	10	60	30	100	4
3	{1, 2, 4}	10	50	30	90	

以4为中间点计算

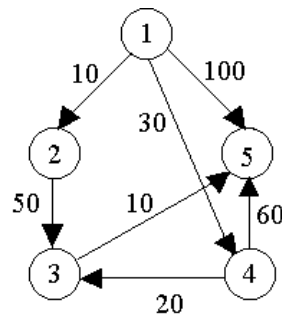
$$\begin{aligned}
 d(1, 5) &= dist(4) + c(4, 5) \\
 &= 30 + 60 = 90 < 100
 \end{aligned}$$



j	1	2	3	4	5
p	-	1	4	1	4

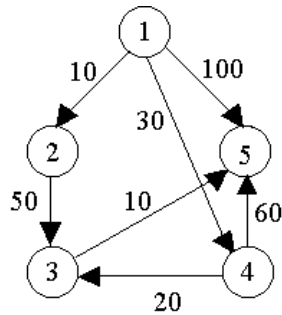
迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2
2	{1, 2}	10	60	30	100	4
3	{1, 2, 4}	10	50	30	90	3
			选择顶点3			

j	1	2	3	4	5
p	-	1	4	1	4



迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2
2	{1, 2}	10	60	30	100	4
3	{1, 2, 4}	10	50	30	90	3
4	{1, 2, 4, 3}	10	50	30		

源顶点1到顶点2、4、3的最短
特殊路径长度不变。



j	1	2	3	4	5
p	-	1	4	1	3

更新 $p(5) = 3$

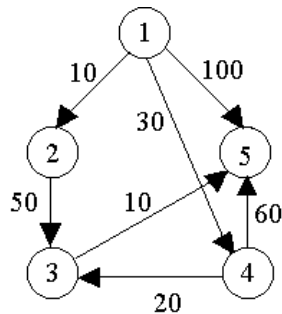
迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2
				30	100	4
				30	90	3
4	{1, 2, 4, 3}	10	50	30	60	5

以3为中间点计算

$$\begin{aligned}
 d(1, 5) &= dist(3) + c(3, 5) \\
 &= 50 + 10 = 60 < 90
 \end{aligned}$$

选择顶点5

Dijkstra算法



j	1	2	3	4	5
p	-	1	4	1	3

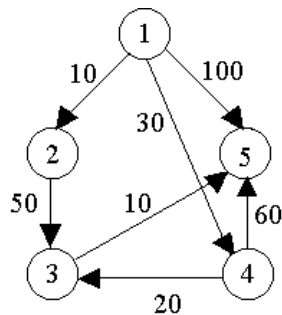
迭代	S	$dist(2)$	$dist(3)$	$dist(4)$	$dist(5)$	u
1	{1}	10	∞	30	100	2
2	{1, 2}	10	60	30	100	4
3	{1, 2, 4}	10	50	30	90	3
4	$S = V$, 计算结束		节点1到其余节点的最短距离			5
5	{1, 2, 4, 3, 5}	10	50	30	60	

Q: 源顶点1到顶点5的最短路径?

j	1	2	3	4	5
p	-	1	4	1	3
$dist$	-	10	50	30	60

最短路径值: 60

最短路径: 1-4-3-5



Dijkstra算法

1) 设计合适的数据结构:

- 带权邻接矩阵`linkMatrix`, 记录输入有向图的权值
- 数组`lowLR`记录从源到其它顶点的最短受限路径长度
- 数组`preV`记录最短路径中的前驱顶点, 假设 $preV[x] = u$, 则在 x 的最短路径中, u 是 x 的前驱顶点

2) **初始化**: 把源加入集合 S 中, 即 $S = \{v\}$, 对于 $V-S$ 中的所有顶点 x , 设置 $lowLR[x] = linkMatrix[v][x]$, $preV[x] = v$ (如果有边相连)

3) **贪心选择**: 在集合 $V-S$ 中依照贪心策略寻找`lowLR[x]`最小的顶点 t , 即

$$lowLR[t] = \min\{lowLR[x] \mid x \in (V - S)\}$$

4) **更新过程**: 将顶点 t 加入 S 中, 同时更新集合 $V-S$, 以及集合中顶点的最短特殊路径:

$$\begin{aligned} lowLR[x] &= lowLR[t] + linkMatrix[t][x] \\ st. \quad lowLR[x] &> lowLR[t] + linkMatrix[t][x] \end{aligned}$$

```

#define INF 0x03F3F3F3F //预定义的充分大的值
#define MaxV 100
int preV[MaxV]; //最短路径树中的前驱结点信息表
int visited[MaxV]; //结点是否加入S的标记表, 0 是未加入, 1 为已加入
void Dijkstra(int linkMatrix[][MaxV], int lowLR[MaxV], int numV, int beginV) {
    int i, j, min, newCost;
    memset(visited, 0, sizeof(visited)); //初始化, 所有结点未加入
    //开始结点beginV加入S
    visited[beginV] = 1;
    for (i=0; i<numV; i++) {
        lowLR[i] = linkMatrix[beginV][i];
        preV[i] = beginV;
    }
    lowLR[beginV] = 0;
    preV[beginV] = -1; // 树根的标记
    int selectV = beginV;
    for (i=1; i<numV; i++) {
        for (j=0; j<numV; j++) { //更新特殊路径
            newCost = lowLR[selectV]+linkMatrix[selectV][j];
            if (visited[j]==0 && newCost<lowLR[j]) {
                lowLR[j] = newCost;
                preV[j] = selectV;
            }
        }
        min = INF;
        for (j=0; j<numV; j++) //贪心选择最短特殊路径
            if (visited[j] == 0 && lowLR[j] < min) {
                min = lowLR[j];
                selectV = j;
            }
        visited[selectV] = 1; //更新被选择顶点的状态
    }
}

```

$$O(n^2)$$

正确性证明

命题：设 $G = \langle V, E, W \rangle$ 是有向带权图，且任意边的权重为**非负数**，即 $w(e) \geq 0$ 。另 $v \in V$ 是源顶点， $short(x)$ 是从 v 到 x 的**最短路径长度**。 S 是当前最短路径树中的顶点集，对于 $V - S$ 中的任意顶点 u ， $dist(u)$ 表示从源顶点 v 到 u 的**最短受限路径长度**。那么对于任意正整数 k ，当Dijkstra算法执行到第 k 步时， $\forall i \in S$ ，有 $dist(i) = short(i)$ 。

证明（数学归纳法）： 对执行步数 k 进行归纳

基础步： $k = 1$ ， $S = \{v\}$ 时，有 $dist(v) = short(v) = 0$

归纳步：

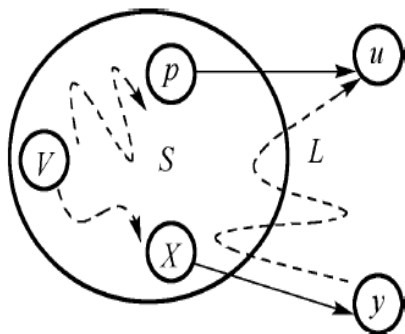
□ 假设对于 k 命题为真，即算法前 k 步选择的结点 i 都有 $dist(i) = short(i)$

□ 算法在第 $k + 1$ 步，选择了顶点 u ，需证明： $dist(u) = short(u)$

□ **反证法，假设存在更短路径** $L(v \dots x \dots y \dots u)$ ，并设 y 是路径 L 中**第一个非 S 中的顶点**

$$\left. \begin{array}{l} d(y, u) > 0 \\ d(L) = dist(y) + d(y, u) \end{array} \right\} \left. \begin{array}{l} dist(y) < d(L) \\ d(L) < dist(u) \end{array} \right\} \boxed{dist(y) < dist(u)}$$

与前提矛盾！



Dijkstra算法

- **Dijkstra 算法**（迪杰斯特拉算法）是解决**单源最短路径问题**的经典算法，由荷兰计算机科学家 Edsger W. Dijkstra 于1956年提出。
- **核心目标：**在带权有向图或无向图中，找到从一个起点到所有其他节点的最短路径。
- **限制条件：**图中所有边的权重必须非负（若存在负权边，需改用 Bellman-Ford 算法）。

关于 Dijkstra 算法的限制条件，以下哪项是正确的？

- ☐ A 可以处理任意权重的边（包括负权边）
- ☐ B 仅适用于无向图
- ☒ C 无法处理存在负权边的图
- ☐ D 只能在无环图中使用

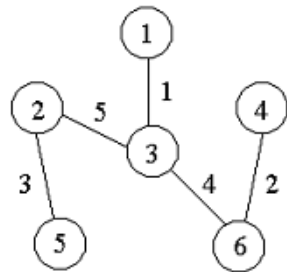
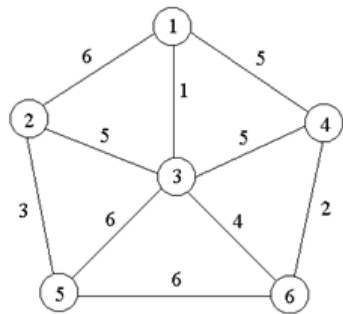
提交

最小生成树问题

问题描述： 设 $G = (V, E)$ 是无向**连通带权图**。E中每条边 (v, w) 的权为 $C(v, w)$ 。如果 G 的子图 G' 是一棵包含 G 的所有顶点的树，则称 G' 为 G 的生成树。生成树上各边权的总和称为该生成树的耗费。在 G 的所有生成树中，耗费最小的生成树称为 G 的最小生成树。请设计算法计算 G 的最小生成树的权重总和。

输入： 多组测试数据。每组测试数据的第一行输入图 G 中顶点的个数 n ($n < 1000$)；后续 n 行输入图 G 的权值矩阵 C 。第 i 行第 j 列 ($j \leq i$) 的值表示边 (i, j) 的权值，如果为-1表示从第 i 个顶点到第 j 个顶点没有边相连。

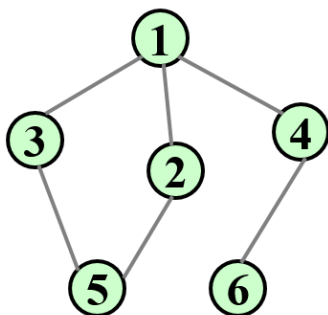
输出： 最小生成树权重之和，每组测试数据输出一行。



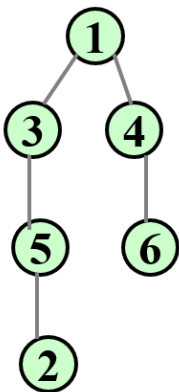
图的生成树

定义：如果连通图 G 的一个子图是一棵包含 G 的所有顶点的树，则该子图称为 G 的**生成树** (Spanning Tree)。

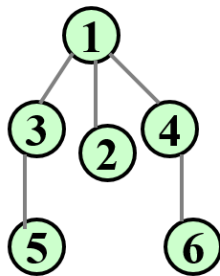
- 生成树是连通图的包含图中的所有顶点的 **极小连通子图**
- 图的生成树不惟一，从不同的顶点出发进行遍历，可以得到不同的生成树
- 常用的生成树算法有DFS生成树、BFS生成树、PRIM 最小生成树和Kruskal 最小生成树算法



G4



图G4的DFS生成树

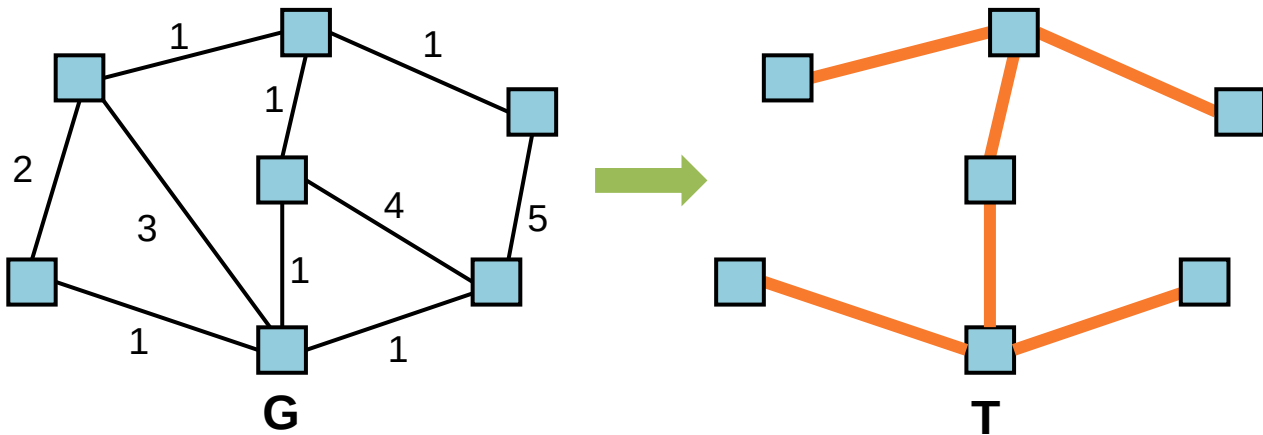


图G4的BFS生成树

最小生成树

定义： 对于连通的带权图 $G=(V, E)$ ，其生成树也是带权的，生成树 $T=(V, TE)$ 各边的权值总和称为该树的权 $W(T)$ ， TE 表示 T 的边集， $C(u, v)$ 表示边 (u, v) 的权重。权最小的生成树称为 G 的**最小生成树**（Minimum Spanning Tree, MST）。

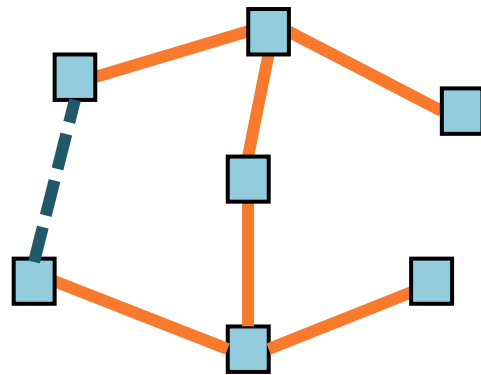
$$W(T) = \sum_{(u,v) \in TE} C(u, v)$$



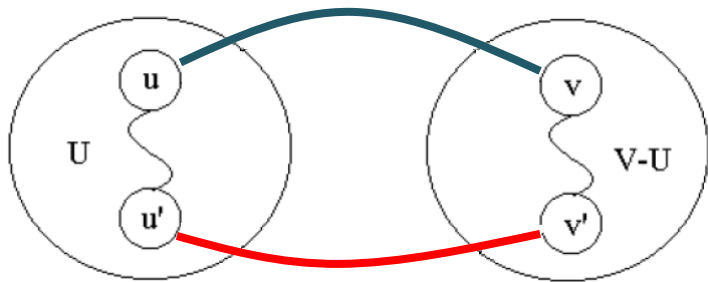
最小生成树性质

□ 生成树有 $|V|$ 个顶点, $|V|-1$ 条边

□ 生成树中**增加**一条边则得到一个**回路**, 回路中**删除**任何一条边又得到一棵生成树



□ 设 $G = (V, E)$ 是连通带权图, U 是 V 的真子集。如果 $(u, v) \in E$, 其中 $u \in U, v \in (V - U)$, 称之为 **跨边**, 且在所有这样的边中, (u, v) 的权 $C(u, v)$ **最小**, 那么一定存在 G 的一棵最小生成树, 它包含边 (u, v) 。



在任意一个环路中，若某条边的权重大于该环路上所有其他边的权重，则这条边一定不属于任何最小生成树。请问该说法正确否？

☐ A 不正确

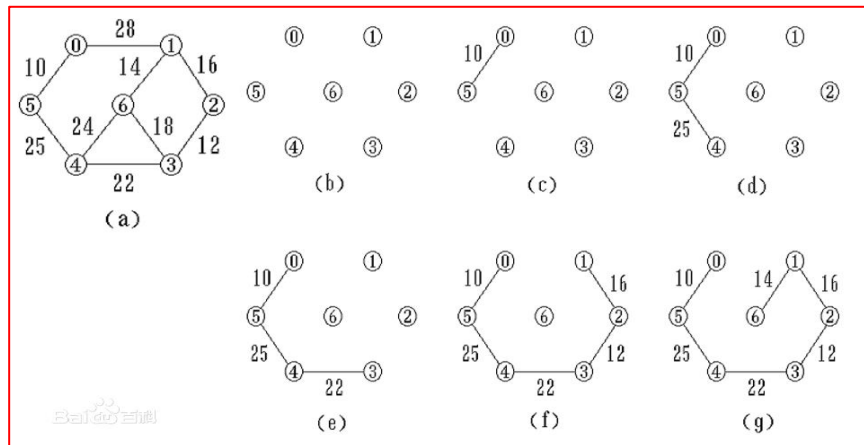
☒ B 正确

提交

最小生成树-求解算法

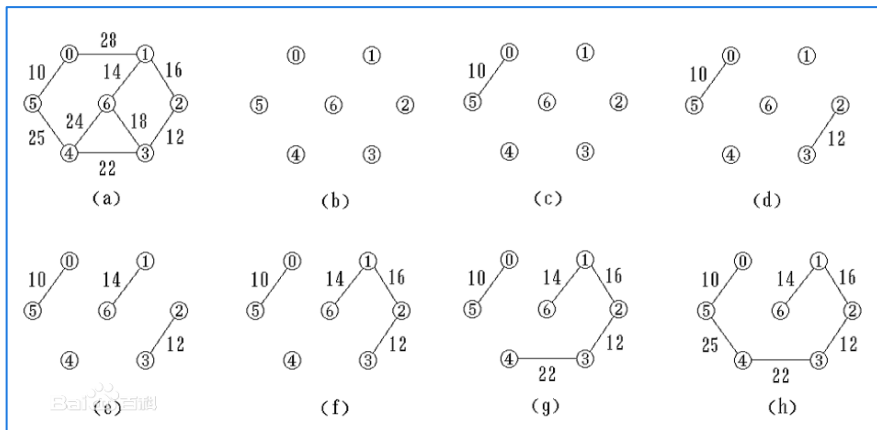
Prim算法:

- 针对图中的**顶点**设计贪心策略，逐步**添加顶点/边**构造最小生成树
- **策略**: 选择**最短跨边**



Kruskal算法:

- 针对图中的**边**设计贪心策略，逐步**合并分支**构造最小生成树
- **策略**: 选择**最短边**合并两个**不连通分支**



最小生成树-Prim算法

给定无向带权图 $G = \langle V, E \rangle$ ，逐步构造最小生成树： $MST = (U, TE)$ ，其中 $U \subseteq V$ ， $TE \subseteq E$ 。图 G 中的顶点集合 V 划分为**两部分**：集合 U 和 $V-U$ ，其中 **U 是已加入MST**中的顶点集合，而 **$V-U$ 是MST之外的**顶点集合。

1. 初始化： $U = \{x\}$ ，其中 x 为集合 V 中的任一结点（起始点）， $TE = \{\}$ ；
2. 循环直到 $U = V$ ：
 - **贪心选择**： **$i \in U, j \in V - U$** ，且边 (i, j) 是连接 U 和 $V-U$ 的所有边中的**最短边**
 - 把顶点 j 加入集合 U
 - 边 (i, j) 加入集合 TE

最小生成树 — Prim算法

□ 设计合适的**数据结构**:

- 带权邻接矩阵linkMatrix, 记录输入无向图的权值
- 数组lowCost**最短跨边**, lowCost[j]表示满足 $j \in V - U, i \in U$ 的所有边 (i, j) 中的最小值
- 数组closest记录上述最短边 (i, j) 中的顶点标号 i , 它们满足 $lowCost[j] = C(j, closest[j])$ 。

□ **初始化**: 把顶点1加入U中, 得 $U = \{1\}$, $TE = \{\}$, 并**初始化**数组 closest 和 lowCost

□ **贪心选择**: 在集合 $V - U$ 中寻找使得**lowCost最小**的顶点 **t**, 将 t **加入**集合U, 边 $(t, closest[t])$ **加入**集合TE

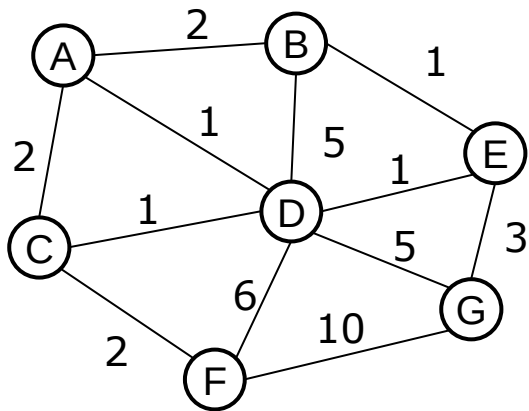
□ **更新过程**: 对 $V - U$ 中的所有顶点 k , 按下述规则**更新数组** closest 和 lowCost:

$if (linkMatrix[t][k] < lowCost[k])$

$lowCost[k] = linkMatrix[t][k]; closest[k] = t$

Prim算法运行实例

输入连通图G:

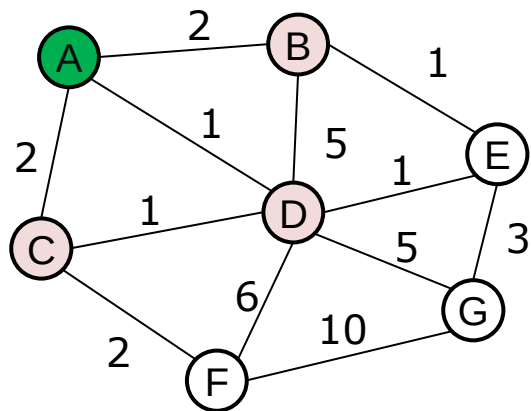


$U = \{\}$

$TE = \{\}$

vertex	visited?	lowCost	closest
A	0	INF	-1
B	0	INF	-1
C	0	INF	-1
D	0	INF	-1
E	0	INF	-1
F	0	INF	-1
G	0	INF	-1

Prim算法运行实例

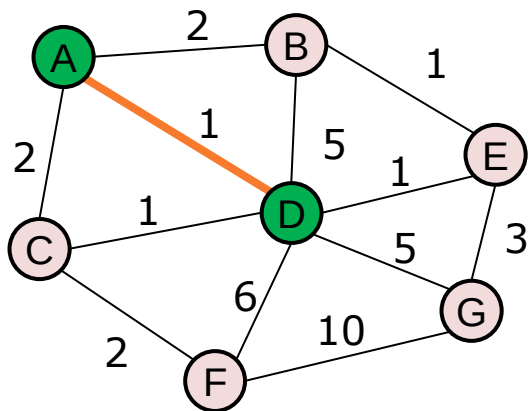


$$U = \{A\}$$

$$TE = \{\}$$

vertex	visited?	lowCost	closest
A	1	0	-1
B	0	2	A
C	0	2	A
D	0	1	A
E	0	INF	-1
F	0	INF	-1
G	0	INF	-1

Prim算法运行实例

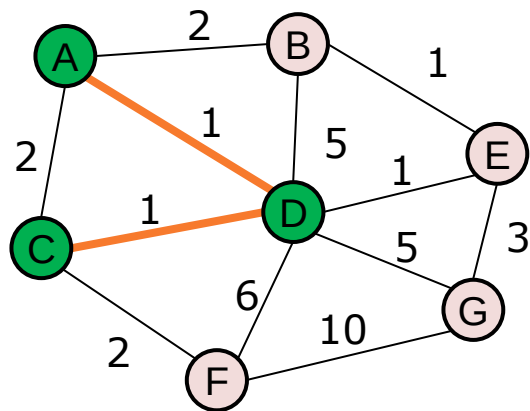


$$U = \{A, D\}$$

$$TE = \{(A, D)\}$$

vertex	visited?	lowCost	closest
A	1	0	-1
B	0	2	A
C	0	2 1	A D
D	1	1	A
E	0	1	D
F	0	6	D
G	0	5	D

Prim算法运行实例

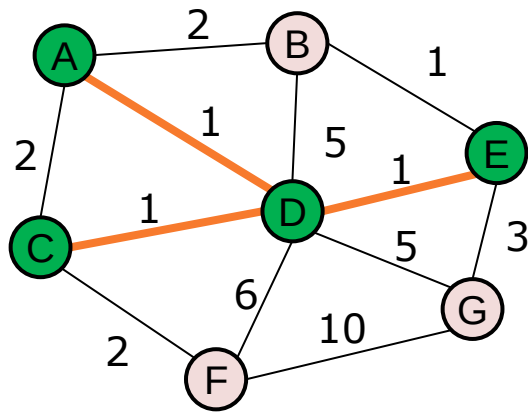


$$U = \{A, D, C\}$$

$$TE = \{(A, D), (D, C)\}$$

vertex	visited?	lowCost	closest
A	1	0	-1
B	0	2	A
C	1	2 1	A D
D	1	1	A
E	0	1	D
F	0	6 2	D C
G	0	5	D

Prim算法运行实例

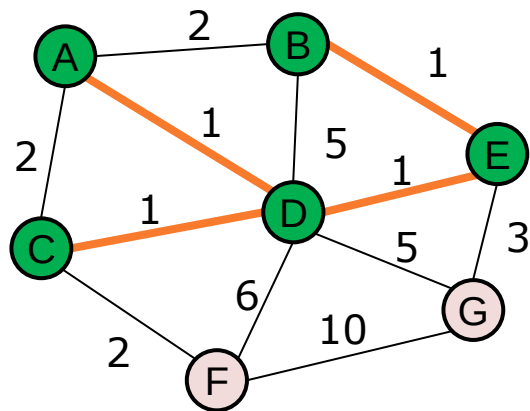


$$U = \{A, D, C, E\}$$

$$TE = \{(A, D), (D, C), (D, E)\}$$

vertex	visited?	lowCost	closest
A	1	0	-1
B	0	2 1	A E
C	1	2 1	A D
D	1	1	A
E	1	1	D
F	0	6 2	D C
G	0	5 3	D E

Prim算法运行实例

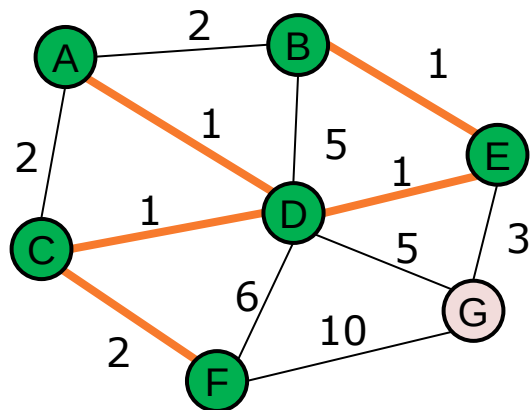


$$U = \{A, D, C, E, B\}$$

$$TE = \{ (A, D), (D, C), (D, E), (E, B) \}$$

vertex	visited?	lowCost	closest
A	1	0	-1
B	1	2 1	A E
C	1	2 1	A D
D	1	1	A
E	1	1	D
F	0	6 2	D C
G	0	5 3	D E

Prim算法运行实例

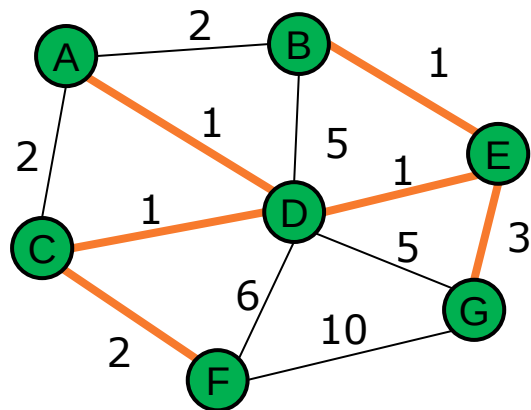


vertex	visited?	lowCost	closest
A	1	0	-1
B	1	2 1	A E
C	1	2 1	A D
D	1	1	A
E	1	1	D
F	1	6 2	D C
G	0	5 3	D E

$$U = \{A, D, C, E, B, F\}$$

$$TE = \{(A, D), (D, C), (D, E), (E, B), (C, F)\}$$

Prim算法运行实例

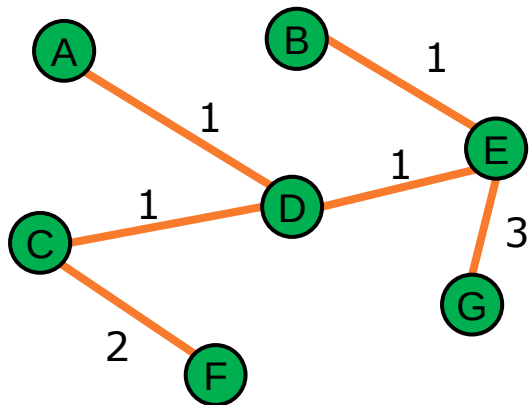


$U = \{A, D, C, E, B, F, G\}$

$TE = \{(A, D), (D, C), (D, E), (E, B), (C, F), (E, G)\}$

vertex	visited?	lowCost	closest
A	1	0	-1
B	1	2 1	A E
C	1	2 1	A D
D	1	1	A
E	1	1	D
F	1	6 2	D C
G	1	5 3	D E

Prim算法运行实例



最小生成树边集合TE:

(A, D) (C, F)
(B, E) (D, E)
(C, D) (E, G)

总权重: $W(TE) = 9$

vertex	visited?	lowCost	closest
A	1	0	-1
B	1	1	E
C	1	1	D
D	1	1	A
E	1	1	D
F	1	2	C
G	1	3	E

```

#define MaxV 1000
#define INF 0x03F3F3F3F //预定义的充分大的值
int prim(int linkMatrix[][MaxV], int numV) {
    int visited[MaxV]= {0}; //结点是否加入MST的标记表, 0是未加入, 1为已加入
    int lowCost[MaxV], closest[MaxV];
    int i, k;    int min, costMST = 0;
    visited[0]=1; //顶点加入MST
    closest[0]=-1; // 树根的标记
    for(i=1; i<numV; i++) { // 初始化
        lowCost[i]= linkMatrix[0][i];
        closest[i] = 0;
    }
    for(i=0; i<numV-1; i++) {
        min=INF;
        int selectV = 0;
        for(k=1; k<numV; k++) { //贪心选择最短的边
            if(!visited[k]&&lowCost[k]<min) {
                min=lowCost[k];
                selectV=k;
            }
        }
        costMST += min; //更新MST树权重
        visited[selectV] = 1;
        for(k=1; k<numV; k++) //更新信息
            if(!visited[k]&&linkMatrix[selectV][k]<lowCost[k]) {
                lowCost[k]=linkMatrix[selectV][k];
                closest[k]=selectV;
            }
        }
    }
    return costMST; }

```

$$O(|V|^2)$$

正确性证明

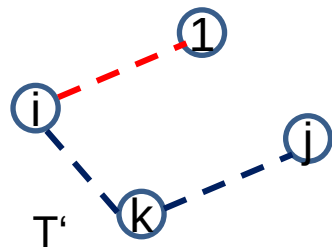
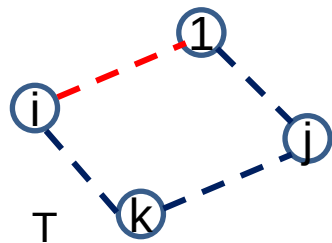
命题：对于任意正整数 k ($k < n$)，**存在**一棵最小生成树包含Prim算法前 k 步选择的边。

证明（数学归纳法）：对执行步数 k 进行归纳

基础步： $k = 1$ ， $U = \{1\}$ 时，存在一棵最小生成树 T 包含边 $(1, i)$ ，该边是所有关联顶点 1 的边中**权重最小**的。

1) 如果边 $(1, i)$ **包含** 在最小生成树 T 中，则命题成立

2) 如果边 $(1, i)$ **未包含** 在最小生成树 T 中，则 $T \cup (1, i)$ 包含回路，用边 $(1, i)$ 替换回路中与顶点 1 相连接的边 $(1, j)$ ，则得到一棵新的生成树 T' ，容易证明： **$W(T') \leq W(T)$**



正确性证明

命题：对于任意正整数 k ($k < n$)，**存在**一棵最小生成树包含Prim算法前 k 步选择的边。

证明（数学归纳法）：对执行步数 k 进行归纳

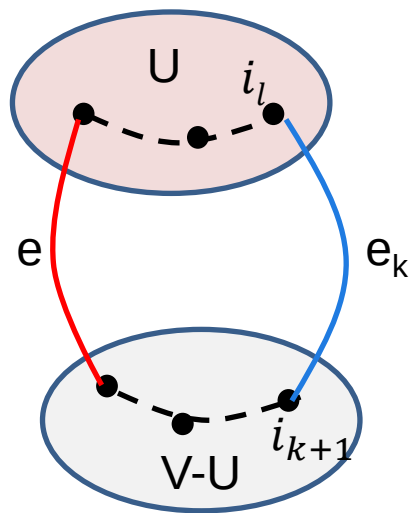
归纳步：

假设算法执行了 $k-1$ 步，生成树的边为 $\{e_1, e_2, \dots, e_{k-1}\}$ ，这些边的 k 个顶点构成集合 U ，图 G 的一棵最小生成树 T 包含这些边。

假设算法**第 k 步**选择了**跨边** $e_k = (i_l, i_{k+1})$ ，其中 $i_l \in U, i_{k+1} \in V - U$

- 1) 如果 T **包含**跨边 e_k ，则命题成立；
- 2) 如果 T **不包含**跨边 e_k ，则 $T \cup e_k$ 包含回路，用 e_k 替换 T 中原来 U 和 $V-U$ 之间的跨边（记为 e ），则得到**新的生成树** T'

$$W(T') \leq W(T)$$



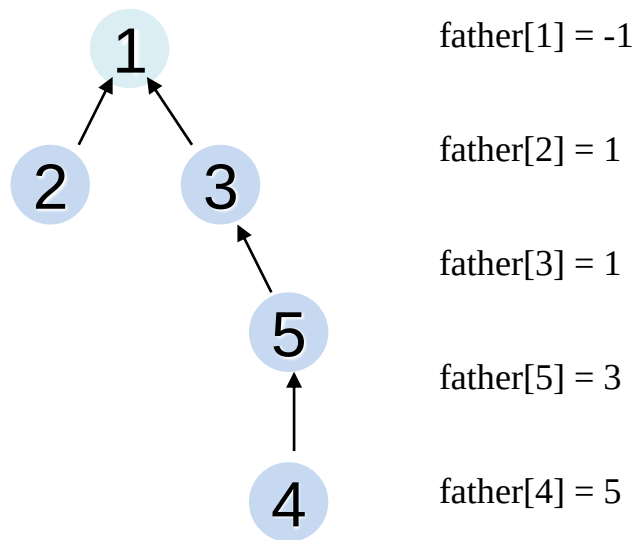
并查集概念

并查集 (Disjoint set或者Union-find set) 是一种**树型**的数据结构，常用于处理一些不相交集合并 (Disjoint Sets) 的合并及查询问题。它包含两种基本操作：

- **查询 (Find)**：查询某个元素在哪个集合里
- **合并 (Union)**：合并两个集合

集合怎么表示？集合怎么存储？

- 每个集合选定一个固定的元素作为该集合的代表，称为**代表元素**，代表元素则用于标识整个集合
- 每一个集合用树表示，树中的每一个节点保存着到其**父节点**的引用，每个集合的代表元素即是集合的根节点。
- **双亲表示法**：用数组father[]存储整个并查集



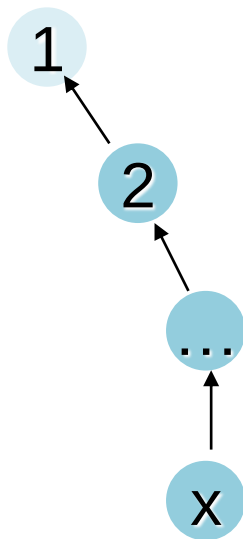
并查集的查询

查询： 给定任意元素 x ，返回它所在集合的标号。

方法： 根据其父结点的引用**向根结点遍历**，到达树根并返回代表元素。

```
int Find(int x) {  
    int p = x;  
    while( father[p] > 0 )  
        p = father[p];  
    return p; //集合的代表元素  
}
```

$O(n)$



并查集的合并

合并： 给定两个集合中的元素 x 和 y ，把这两个集合进行合并

方法： 找到 x 和 y 所在集合的根结点，将一棵树的根连接到另一棵树的根



- 合并1和2
- 合并1和3
- 合并5和4
- 合并5和3

$\text{father}[\text{yRoot}] = \text{xRoot}$
or
 $\text{father}[\text{xRoot}] = \text{yRoot}$

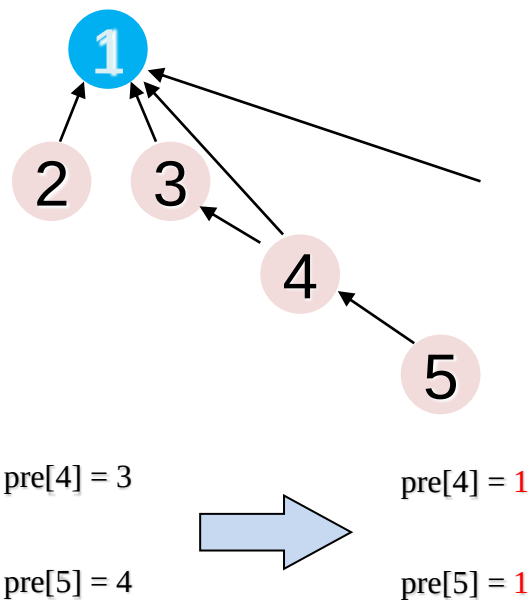
```
int Union(int x, int y)
{
    int xRoot = Find(x);
    int yRoot = Find(y);
    father[yRoot] = xRoot;

    return xRoot; //代表元素
}
```

并查集优化策略（一）

路径压缩是一种在执行“查询”时**扁平化树结构**的方法，为后续查询操作加速。

- 路径上的每个节点都**直接连接到根上**;
- **递归地**访问当前结点到树根的路径，改变每一个结点的引用到根节点。

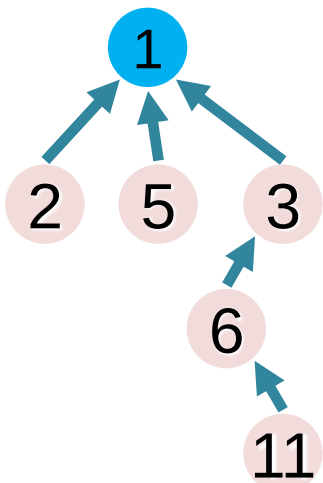


```
int Find(int x) {  
    int p = x;  
  
    while( father[p] > 0 )  
        p = father[p]; //找到树根  
  
    while (father[x] != p &&  
           father[x] > 0) {  
        int temp = father[x];  
        father[x] = p; //指向树根  
        x = temp;  
    }  
    return p;  
}
```

并查集优化策略（二）

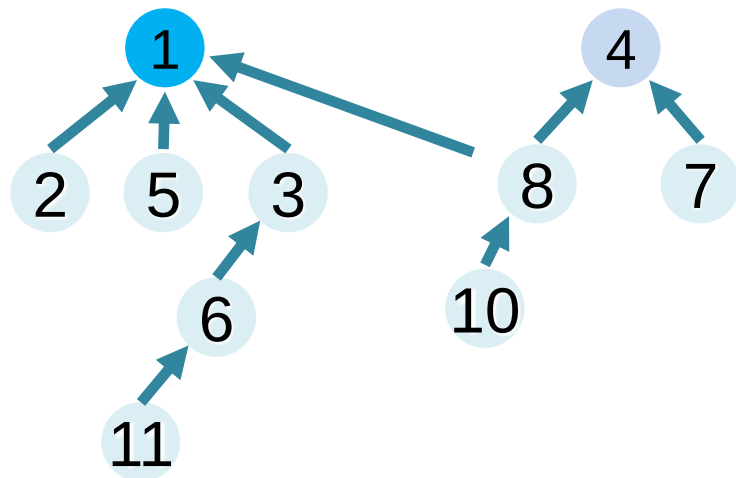
按秩合并即总是将**更浅的树连接至更深的树上**，避免**增加合并树的秩**

- “秩”表示集合对应树的深度，单个元素的树的秩定义为0；
- 当两棵秩同为 r 的树联合时，它们的秩变为 $r + 1$ ；
- 为了方便比较，把集合秩的相反数存储在**根节点father[root]**。



`father[1] = -3;`

并查集优化策略（二）



```
void Union(int x, int y){
    int xRoot = Find(x);
    int yRoot = Find(y);

    if( xRoot == yRoot ) return ;

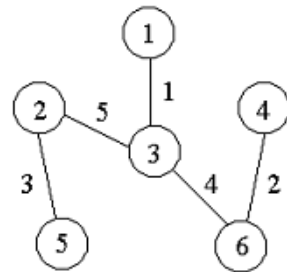
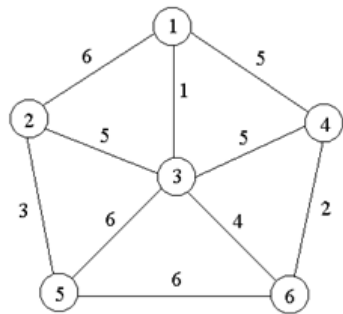
    if( -father[xRoot] > -father[yRoot] )
        father[yRoot] = xRoot;
    else {
        if(father[xRoot] == father[yRoot])
            father[yRoot] -= 1; //相等时
        father[xRoot] = yRoot;
    }
}
```

最小生成树问题

问题描述: 设 $G = (V, E)$ 是无向连通带权图。 E 中每条边 (v, w) 的权为 $C(v, w)$ 。如果 G 的子图 G' 是一棵包含 G 的所有顶点的树，则称 G' 为 G 的生成树。生成树上各边权的总和称为该生成树的耗费。在 G 的所有生成树中，耗费最小的生成树称为 G 的最小生成树。请设计算法计算 G 的最小生成树的权重总和。

输入: 多组测试数据。每组测试数据的第一行输入图 G 中顶点的个数 n ($n < 1000$)；后续 n 行输入图 G 的权值矩阵 C 。第 i 行第 j 列 ($j \leq i$) 的值表示边 (i, j) 的权值，如果为-1表示从第 i 个顶点到第 j 个顶点没有边相连。

输出: 最小生成树权重之和，每组测试数据输出一行。



最小生成树- Kruskal算法

- 给定无向带权图 $G = (V, E)$, $V = \{1, 2, \dots, n\}$ 。设最小生成树 $MST = (V, TE)$, 该树的初始状态为**只有n个顶点而无边**的**非连通图** $T = (V, \{\})$, Kruskal算法将这 n 个顶点看成是 n 个**孤立的连通分支**;
- **贪心选择**: 在边集 E 中选择**权值最小**的边 (i, j) , 如果将边 (i, j) 加入 TE 中**不产生回路**, 则将边 (i, j) 加入 TE , 即用边 (i, j) 将 T 中的两个连通分支合并成一个联通分支; 否则**舍弃**它。循环此过程, 直至所有顶点都在一个联通分支。

Kruskal算法俗称**避环法**, 该算法的实现关键是在加入边时**避免出现环路**用并查集来实现连通分支查找和合并的相关操作

最小生成树- Kruskal算法

1. **初始化**：把图G中的所有边按照权值**从小到大**排序，MST的边集 TE 初始化为空集。
每一个顶点初始化为一个孤立的连通分支，对应并查集中的一个子集合。
2. 在E中寻找**权值最小**的边 (i, j) 。
3. 如果顶点 i 和 j 位于两个**不同的集合**
 - 则将边 (i, j) **加入**边集TE
 - 并把顶点 i 和 j 所在的两个子集合并成一个子集
 - 将边 (i, j) 从E中删除
4. 如果连通分支的数目**大于1**，则转步骤2；否则，算法结束。

Kruskal算法运行实例

按权重**递增排列**的边:

1: (A,D) (C,D) (B,E) (D,E)

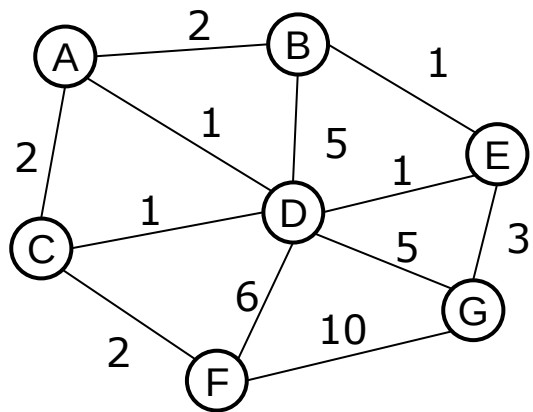
2: (A,B) (C,F) (A,C)

3: (E,G)

5: (D,G) (B,D)

6: (D,F)

10: (F,G)



分支: (A) (B) (C) (D) (E) (F) (G)

TE:

Kruskal算法运行实例

按权重**递增排列**的边:

1: ~~(A,D)~~ (C,D) (B,E) (D,E)

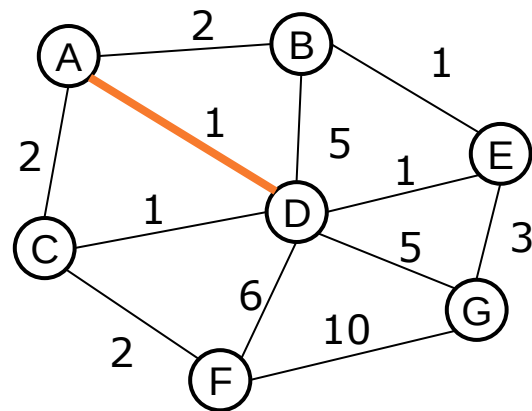
2: (A,B) (C,F) (A,C)

3: (E,G)

5: (D,G) (B,D)

6: (D,F)

10: (F,G)



分支: (A,D) (B) (C) (E) (F) (G)

TE: (A,D)

Kruskal算法运行实例

按权重**递增排列**的边:

1: ~~(A,D)~~ ~~(C,D)~~ (B,E) (D,E)

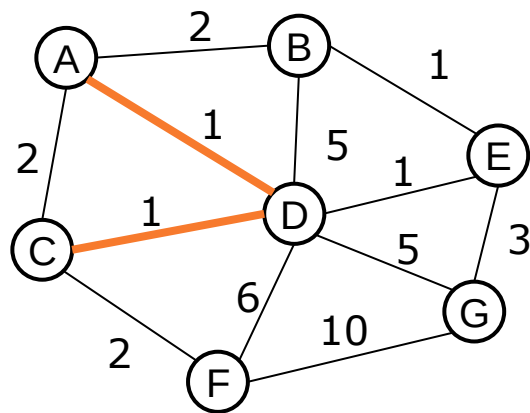
2: (A,B) (C,F) (A,C)

3: (E,G)

5: (D,G) (B,D)

6: (D,F)

10: (F,G)



分支: (A,C,D) (B) (E) (F) (G)

TE: (A,D) (C,D)

Kruskal算法运行实例

按权重**递增排列**的边:

1: ~~(A,D)~~ ~~(C,D)~~ ~~(B,E)~~ (D,E)

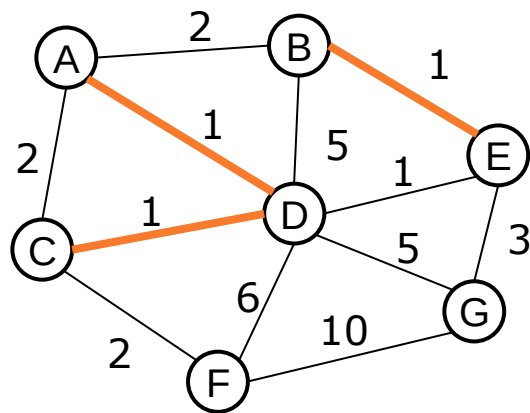
2: (A,B) (C,F) (A,C)

3: (E,G)

5: (D,G) (B,D)

6: (D,F)

10: (F,G)



分支: (A,C,D) (B,E) (F) (G)

TE: (A,D) (C,D) (B,E)

Kruskal算法运行实例

按权重**递增排列**的边:

1: ~~(A,D)~~ ~~(C,D)~~ ~~(B,E)~~ ~~(D,E)~~

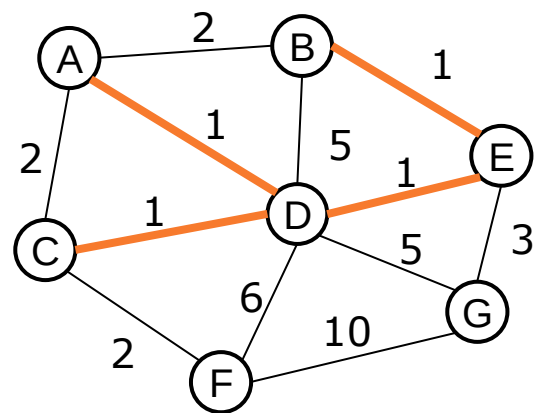
2: (A,B) (C,F) (A,C)

3: (E,G)

5: (D,G) (B,D)

6: (D,F)

10: (F,G)



分支: (A,B,C,D,E) (F) (G)

TE: (A,D) (C,D) (B,E) (D,E)

Kruskal算法运行实例

按权重**递增排列**的边:

1: ~~(A,D)~~ ~~(C,D)~~ ~~(B,E)~~ ~~(D,E)~~

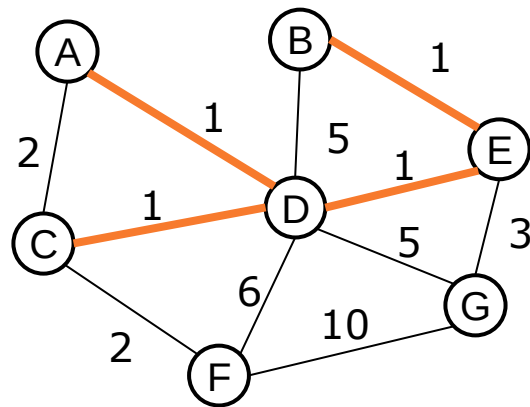
2: ~~(A,B)~~ (C,F) (A,C)

3: (E,G)

5: (D,G) (B,D)

6: (D,F)

10: (F,G)



分支: (A,B,C,D,E) (F) (G)

TE: (A,D) (C,D) (B,E) (D,E)

Kruskal算法运行实例

按权重**递增排列**的边:

1: ~~(A,D)~~ ~~(C,D)~~ ~~(B,E)~~ ~~(D,E)~~

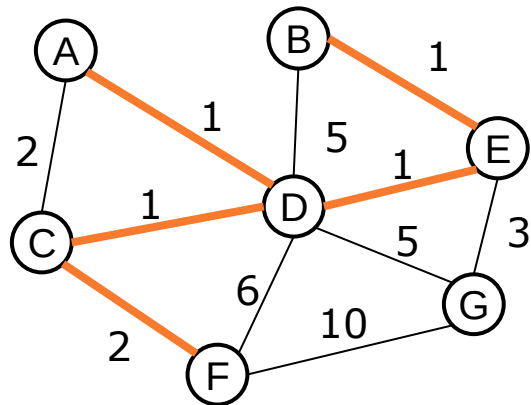
2: ~~(A,B)~~ ~~(C,F)~~ (A,C)

3: (E,G)

5: (D,G) (B,D)

6: (D,F)

10: (F,G)



分支: (A,B,C,D,E,F) (G)

TE: (A,D) (C,D) (B,E) (D,E) (C,F)

Kruskal算法运行实例

按权重**递增排列**的边:

1: ~~(A,D)~~ ~~(C,D)~~ ~~(B,E)~~ ~~(D,E)~~

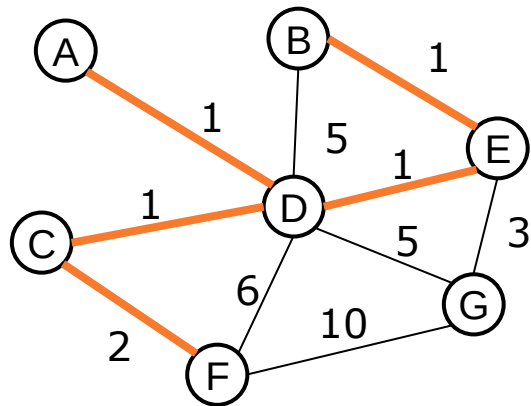
2: ~~(A,B)~~ ~~(C,F)~~ ~~(A,C)~~

3: (E,G)

5: (D,G) (B,D)

6: (D,F)

10: (F,G)



分支: (A,B,C,D,E,F) (G)

TE: (A,D) (C,D) (B,E) (D,E) (C,F)

Kruskal算法运行实例

按权重**递增排列**的边:

1: ~~(A,D)~~ ~~(C,D)~~ ~~(B,E)~~ ~~(D,E)~~

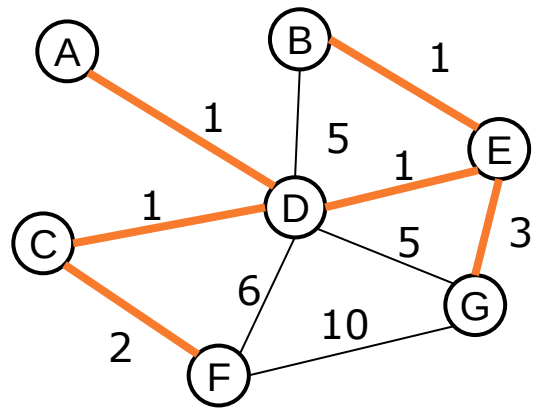
2: ~~(A,B)~~ ~~(C,F)~~ ~~(A,G)~~

3: ~~(E,G)~~

5: (D,G) (B,D)

6: (D,F)

10: (F,G)



分支: (A,B,C,D,E,F,G)

TE: (A,D) (C,D) (B,E) (D,E) (C,F) (E,G)

Kruskal实现

```
int kruskal(node juSet[MaxV],int numV,edge edgeSet[MaxE],int numE) {  
    qsort(edgeSet, numE, sizeof(struct edge), cmp);//将边排序  
    int totalCost = 0, cntE = 0, fatherX, fatherY;  
    for (int i = 0; i < numE; i++) //贪心选择边  
        fatherX = Find_Set(edgeSet[i].x);  
        fatherY = Find_Set(edgeSet[i].y);  
        if (fatherX != fatherY) {  
            Union(fatherX, fatherY);  
            totalCost += edgeSet[i].w;  
            cntE++;  
        }  
        if(cntE == numV-1)//已得到MST  
            return totalCost;  
    }// end for (i  
    return totalCost;  
}
```

$$O(e \log(e))$$

Kruskal算法正确性证明

命题： 给定连通带权图 G ，Kruskal算法的输出 T 是一棵最小生成树。

证明：（交换论证法）

第一步： T 是一棵生成树，因为：

- 1) T 是森林，没有回路
- 2) T 包括所有结点（ G 是连通的、算法性质保障）
- 3) T 是连通的（ G 是连通的、算法性质保障）

第二步： T 是一棵最小生成树：

Kruskal算法正确性证明

命题：给定连通带权图 G ，Kruskal算法的输出 T 是一棵最小生成树。

证明：（交换论证法）

第二步： T 是一棵最小生成树：

证明： e 不包含在 E 中，意味着Kruskal算法考虑 e 时， C 中所有结点已经处于一个分支，也就是说其它边先于 e 被考虑，不要忘记：边是按照权重生序排列和考虑)

假设 $T = (V, E)$ 是Kruskal算法得到的生成树； $T^* = (V, E^*)$ 是一棵最小生成树，

□ 如果 $T = T^*$ ，则命题成立；

□ 如果 $T \neq T^*$ ，则 $E \neq E^*$ ， $W(T^*) \leq W(T)$ ， T 和 T^* 至少存在一条**不相同**的边 e ， $e \in E^*, e \notin E$

✓ 把 e 加入树 T 中，则得到回路 **C**，可以证明在 C 中 e 的权重是**最大的**；

✓ C 中至少存在一条边（记为 f ），连接 $T^* - \{e\}$ 的两个分支，且 $w(f) \leq w(e)$

Kruskal算法正确性证明

命题：给定连通带权图 G ，Kruskal算法的输出 T 是一棵最小生成树。

证明：（交换论证法）

第二步： T 是一棵最小生成树：

不同要素： $T = (V, E)$ 是Kruskal算法的生成树； $T^* = (V, E^*)$ 是最小生成树， $T \neq T^*$ ， $E \neq E^*$ ， $e \in E^*, e \notin E$ ， $f \in E, f \notin E^*$ ， $w(f) \leq w(e)$

替换：把边 f 加入非连通子图 $T^* - \{e\}$ 中，则得到**新生成树** $T_1 = T^* - \{e\} \cup \{f\}$ ，而且 $W(T_1) = W(T^*) - w(e) + w(f) \leq W(T^*)$ ，即 T_1 也是最小生成树，但是 T_1 中**多了一条Kruskal算法选择的边**。

循环执行：我们类似处理 T 和 T_1 的非公共边，得到最小生成树 T_2 ， T_2 与 T 的公共边又增加1条；这个交换过程可以**循环执行，最多 $n-1$ 次**交换得到 T ，满足：

$$W(T^*) \geq W(T_1) \geq \dots \geq W(T_{n-1}) \geq W(T)$$

在一个连通无向图中，顶点集为 $\{A, B, C, D\}$ ，边的权重依次为： $(A,B)=4$, $(A,C)=3$, $(A,D)=5$, $(B,C)=2$, $(B,D)=6$, $(C,D)=1$ 按照Kruskal算法构造最小生成树时，选择的边顺序是？

- ☒ A $(C,D,1) \rightarrow (B,C,2) \rightarrow (A,C,3)$
- ☐ B $(B,C,2) \rightarrow (A,C,3) \rightarrow (C,D,1)$
- ☐ C $(C,D,1) \rightarrow (A,C,3) \rightarrow (B,C,2)$
- ☐ D $(A,C,3) \rightarrow (B,C,2) \rightarrow (C,D,1)$

提交

第四章 小结

□ 贪心算法的基本原理

- 分解为多阶段决策过程
- 设计贪心选择策略
- 合并局部决策得到全局最优解

□ 经典问题

- 活动安排问题
- 哈夫曼编码编码
- 单源最短路径
- 最小生成树



第四章 小结

□ 贪心算法的正确性证明

1) 数学归纳法

针对贪心选择的步骤 (k) 进行归纳, 比如活动安排问题, Dijkstra算法, Prim算法。

2) 交换论证法

从原问题最优解开始, 用贪心策略对最优解进行**改造** (或者**交换**最优解与贪心解的不同要素), 有限步后把最优解改造为贪心解, 而且**最优性保持不变**。比如哈夫曼编码, Kruskal算法。

判断题:

- 贪心法通常以自底向上的方式求解最优解。()

填空题:

若采用贪心算法求解问题最优解，必须满足的基本性质为_____和_____。

Dijkstra算法可以正确处理图中存在负权边的最短路径问题。

☐ A 正确

☒ B 错误

提交

Dijkstra算法可以正确处理图中存在负权边的最短路径问题。

☐ A 正确

☒ B 错误

提交

贪心算法与动态规划的主要区别是：

- ☐ A 需要存储子问题解
- ☒ B 通过局部最优推导全局最优
- ☐ C 时间复杂度更高

提交

给定字符集 {A, B, C, D} 的频率分别为 15%、20%、25%、40%。使用哈夫曼编码时，字符B的编码长度是[填空1]个bit。

作答

谢谢 THANKS